Bachelor of Computer Applications (Hons. AI and ML)
Semester- III

Subject: Database Management System

Assignment 5: Database Creation and Table Implementation

By
Name: AKASH P
Reg. No.: 2511021240008

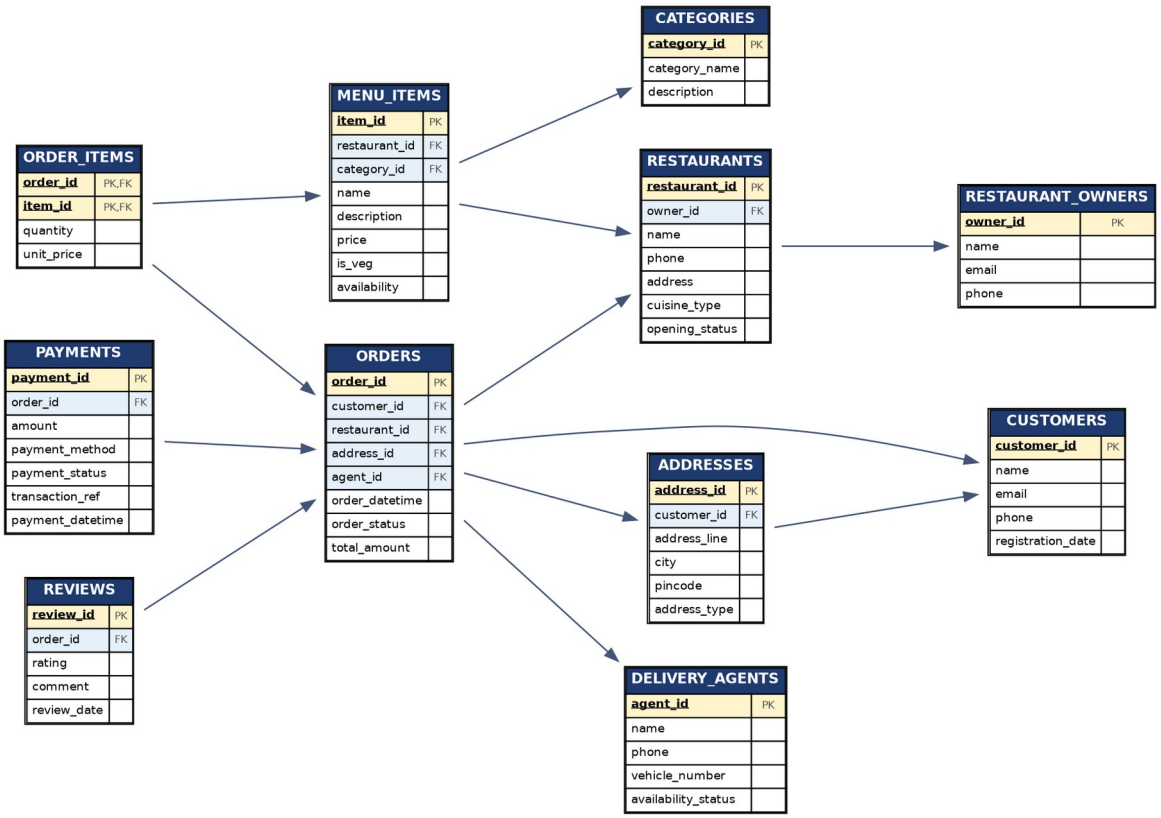
Faculty Name: Renita Joy
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
October 2026

1. Overview

The DDL below creates the database and the 11 tables from Assignments 1 and 2 for the Online Food Delivery System. The statements use MySQL 8.0 syntax. Tables are created in dependency order so referenced tables exist before foreign keys are added.

2. Relationship diagram



3. SQL DDL commands

```
CREATE DATABASE IF NOT EXISTS FoodDelivery;

USE FoodDelivery;

CREATE TABLE Customers (
  customer_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  phone VARCHAR(20) NOT NULL UNIQUE,
  registration_date DATE NOT NULL DEFAULT (CURRENT_DATE)
);

CREATE TABLE Addresses (
  address_id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  address_line VARCHAR(255) NOT NULL,
  city VARCHAR(80) NOT NULL,
  pincode VARCHAR(10) NOT NULL,
  address_type VARCHAR(20) NOT NULL DEFAULT 'Home',
  CONSTRAINT fk_addr_customer FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
  CONSTRAINT chk_address_type CHECK (address_type IN ('Home','Work','Other'))
);

CREATE TABLE Restaurant_Owners (
  owner_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  phone VARCHAR(20) NOT NULL UNIQUE
);

CREATE TABLE Restaurants (
  restaurant_id INT PRIMARY KEY,
  owner_id INT NOT NULL,
  name VARCHAR(150) NOT NULL,
  phone VARCHAR(20),
  address VARCHAR(255) NOT NULL,
  cuisine_type VARCHAR(80),
  opening_status VARCHAR(20) NOT NULL DEFAULT 'Open',
  CONSTRAINT fk_rest_owner FOREIGN KEY (owner_id) REFERENCES Restaurant_Owners(owner_id),
  CONSTRAINT chk_opening_status CHECK (opening_status IN ('Open','Closed'))
);

CREATE TABLE Categories (
  category_id INT PRIMARY KEY,
  category_name VARCHAR(80) NOT NULL UNIQUE,
  description VARCHAR(255)
);

CREATE TABLE Menu_Items (
  item_id INT PRIMARY KEY,
  restaurant_id INT NOT NULL,
  category_id INT NOT NULL,
  name VARCHAR(150) NOT NULL,
  description VARCHAR(255),
  price DECIMAL(8,2) NOT NULL,
  is_veg BOOLEAN NOT NULL DEFAULT TRUE,
  availability BOOLEAN NOT NULL DEFAULT TRUE,
```

```
  CONSTRAINT fk_item_restaurant FOREIGN KEY (restaurant_id) REFERENCES
Restaurants(restaurant_id),
  CONSTRAINT fk_item_category FOREIGN KEY (category_id) REFERENCES Categories(category_id),
  CONSTRAINT uq_item_name UNIQUE (restaurant_id, name),
  CONSTRAINT chk_item_price CHECK (price >= 0)
);

CREATE TABLE Delivery_Agents (
  agent_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  phone VARCHAR(20) NOT NULL UNIQUE,
  vehicle_number VARCHAR(20) NOT NULL UNIQUE,
  availability_status VARCHAR(20) NOT NULL DEFAULT 'Available',
  CONSTRAINT chk_agent_status CHECK (availability_status IN ('Available','Busy','Offline'))
);

CREATE TABLE Orders (
  order_id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  restaurant_id INT NOT NULL,
  address_id INT NOT NULL,
  agent_id INT NULL,
  order_datetime DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  order_status VARCHAR(20) NOT NULL DEFAULT 'Placed',
  total_amount DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  CONSTRAINT fk_orders_customer FOREIGN KEY (customer_id) REFERENCES
Customers(customer_id),
  CONSTRAINT fk_orders_restaurant FOREIGN KEY (restaurant_id) REFERENCES
Restaurants(restaurant_id),
  CONSTRAINT fk_orders_address FOREIGN KEY (address_id) REFERENCES Addresses(address_id),
  CONSTRAINT fk_orders_agent FOREIGN KEY (agent_id) REFERENCES Delivery_Agents(agent_id),
  CONSTRAINT chk_order_status CHECK (order_status IN ('Placed','Accepted','Preparing','Out
for Delivery','Delivered','Cancelled')),
  CONSTRAINT chk_order_total CHECK (total_amount >= 0)
);

CREATE TABLE Order_Items (
  order_id INT NOT NULL,
  item_id INT NOT NULL,
  quantity INT NOT NULL,
  unit_price DECIMAL(8,2) NOT NULL,
  PRIMARY KEY (order_id, item_id),
  CONSTRAINT fk_oi_order FOREIGN KEY (order_id) REFERENCES Orders(order_id),
  CONSTRAINT fk_oi_item FOREIGN KEY (item_id) REFERENCES Menu_Items(item_id),
  CONSTRAINT chk_oi_quantity CHECK (quantity > 0),
  CONSTRAINT chk_oi_price CHECK (unit_price >= 0)
);

CREATE TABLE Payments (
  payment_id INT PRIMARY KEY,
  order_id INT NOT NULL UNIQUE,
  amount DECIMAL(10,2) NOT NULL,
  payment_method VARCHAR(20) NOT NULL,
  payment_status VARCHAR(20) NOT NULL DEFAULT 'Pending',
  transaction_ref VARCHAR(60) UNIQUE,
  payment_datetime DATETIME NULL,
  CONSTRAINT fk_pay_order FOREIGN KEY (order_id) REFERENCES Orders(order_id),
  CONSTRAINT chk_pay_amount CHECK (amount >= 0),
```

```
  CONSTRAINT chk_pay_method CHECK (payment_method IN ('UPI','Card','Net
Banking','Wallet','Cash on Delivery')),
  CONSTRAINT chk_pay_status CHECK (payment_status IN
('Pending','Paid','Failed','Refunded')),
  CONSTRAINT chk_paid_date CHECK (payment_status <> 'Paid' OR payment_datetime IS NOT NULL)
);

CREATE TABLE Reviews (
  review_id INT PRIMARY KEY,
  order_id INT NOT NULL UNIQUE,
  rating TINYINT NOT NULL,
  comment VARCHAR(500),
  review_date DATE NOT NULL DEFAULT (CURRENT_DATE),
  CONSTRAINT fk_review_order FOREIGN KEY (order_id) REFERENCES Orders(order_id),
  CONSTRAINT chk_rating CHECK (rating BETWEEN 1 AND 5)
);
```

4. Implementation notes

- Customers, Restaurant_Owners, Categories, and Delivery_Agents are master tables.
- Addresses references Customers; Restaurants references Restaurant_Owners.
- Menu_Items references Restaurants and Categories.
- Orders references Customers, Restaurants, Addresses, and Delivery_Agents. The agent_id column is nullable because an agent is assigned only after the order is accepted.
- Order_Items uses a composite primary key and two foreign keys to resolve the many-to-many relationship between Orders and Menu_Items; unit_price stores the price at the time of ordering.
- Payments and Reviews reference Orders, with UNIQUE(order_id) enforcing the one-to-one rules from the original design.
- CHECK constraints control status values, non-negative prices and amounts, positive quantity, and ratings from 1 to 5.